ICTCE04

International College
Rangsit University
52/347 Muang-Ake, Phaholyothin Road, Lak-Hok, Muang, Pathumthani 12000, Thailand
T. (662) 997-2200-30 Ext. 4012-4020, F. (662) 997-2200-30 Ext. 4016
www.rsuip.org

Cooperative Education Liaison:
Miss Srinuan Ketkaew
T. (662) 997-2222 Ext. 4012 E-mail: srinuan@rsu.ac.th

Weekly Progress Report Form

Name - Mr. Sai Swam Wan Hline, Student ID - 6609216

Tel. 0649435041, E-mail - saiswamwan.h66@rsu.ac.th,

Company Name /Employer Name - **HTZKKL Technologies Co.,Ltd.**

Reporting Period: *Week No.6, October 2026, 7/10/2026 - 13/10/2026, Total 40 Hours*

1. Name of project/task/issue/topic of student's responsibility.

(List each of the performed tasks.)

Project: Multi-Tenant School Intranet & Academic Management Platform

Task 1: Architecture and implementation of Custom School Payment Channels (dynamic configuration for mobile wallets such as KBZPay, WavePay, CB Pay, and local bank transfers).

Task 2: Development of the Financial Analytics Dashboard & Aging Receivables Engine (/finance/analytics) with overdue payment aging buckets and collection leaderboards.

Task 3: Architecture of the Public Marketing Landing Page (/) featuring 12 module showcases, 5 stakeholder role previews, and bilingual (English/Myanmar) presentation.

Task 4: Refactoring client-side route navigation and public path authentication guards to resolve 401 Unauthorized redirect loops on public pages.

Task 5: Construction of financial analytics test suite (test_finance_analytics.py) and SEO metadata optimization.

2. Summary of the performed tasks.

(Describe the type of work that the student has accomplished.)

During the sixth week of the project:

Dynamic Payment Channels: Extended the financial configuration schema (PaymentChannelsConfig) to support arbitrary custom payment channels (CustomPaymentChannel). School administrators can now dynamically define tenant-specific bank accounts and digital mobile wallets (e.g. KBZPay, WavePay, AYA Pay) with custom instructions and account numbers, which are automatically reflected in student fee payment modals and receipt templates.

Financial Analytics & Aging Receivables:

Designed the backend analytics service in app/modules/finance/service.py to aggregate school revenue metrics, monthly cash flow trends, payment method breakdowns, and fee category distribution.

Built an Aging Receivables algorithm classifying unpaid invoices into standard accounting delinquency tiers (Current, 1–30 days, 31–60 days, 61–90 days, and 90+ days overdue).

Rendered visual charts and class fee collection leaderboards across /dashboard/finance and the central /dashboard/overview page.

Public Landing Page & UI Components: Created a high-converting, fully bilingual marketing landing page at / comprising modular sections: dynamic Hero, platform metrics bar, 12-module feature grid, 5 user-role cards (Super Admin, School Admin, Teacher, Student, Parent), interactive FAQ accordion, and call-to-action banners.

Authentication Interceptor & Routing Hardening:

Implemented intelligent root routing: authenticated users are directed straight to their respective role-based dashboard, while public visitors view the marketing portal.

Patched frontend apiFetch and token refresh interceptors with explicit PUBLIC_PATHS checks and skipRedirect guards, preventing background 401 responses from prematurely bouncing unauthenticated visitors back to /login.

Testing & QA Verification: Added comprehensive test coverage in tests/test_finance_analytics.py and tests/test_mmk_payments_and_slip_verification.py with 120/120 tests passing (100% pass rate).

3. Application of concepts, theories, knowledge, or experiences that applied to cooperative education.

Financial Accounting Principles & Aging Analysis: Applied accounting standards for accounts receivable (A/R) aging schedules to quantify school credit risk and track tuition collection rates across time brackets.

Data Aggregation & Statistical Summaries: Utilized SQL aggregation queries (func.sum(), func.count(), and grouping) within async SQLAlchemy to calculate multi-tenant financial KPIs with minimal database overhead.

Frontend Routing & Navigation Guards: Applied client-side single-page application (SPA) routing patterns in Next.js, managing authenticated session hydration vs. anonymous public route access.

Search Engine Optimization (SEO) & Web Performance: Applied modern web metadata standards (Open Graph protocol and Twitter Card meta tags) and component lazy-loading to optimize initial page load performance.

Financial Schema Flexibility: Modeled polymorphic JSON configuration payloads to enable dynamic tenant-level payment channels without requiring database schema migrations for each new banking partner.

4. Problems/obstacles encountered during the work.

(Provide details on how to solve the problems/obstacles.)

Problem 1: 401 Interceptor Forcing Public Landing Page Visitors to /login:

Detail: The global apiFetch wrapper was configured to automatically redirect any 401 Unauthorized response to the /login screen. When anonymous users loaded public landing pages that made optional status or version checks, they were forcefully redirected to /login, making the public landing page inaccessible.

Solution: Refactored frontend/services/api.ts by defining a whitelist of public routes (PUBLIC_PATHS = ['/', '/login']) and implementing a shouldRedirectToLogin() guard along with a skipRedirect: true flag for background preflight calls.

Problem 2: Performance Bottlenecks in Dynamic Receivables Calculation:

Detail: Iterating over thousands of invoice records in Python memory to calculate aging day brackets caused high latency on the financial analytics endpoint.

Solution: Shifted date-difference filtering and aggregation directly into the SQL query level using SQL CASE statements and date arithmetic, reducing API response times from over 1.2s to under 45ms.



(……Nay Min.....)

Job Supervisor

Date ……13/10/2026.....



(….Sai Swam Wan Hline.....)

Student

Date ……13/10/2026.....